Examen Git, GitHub Pages y GitHub Actions

Resumen de comandos explicados — usuario de GitHub: **jaisormar**

0. Antes de empezar (no puntúa)

`git config --global user.name "Nombre Apellido"`	Tu nombre en los commits.
`git config --global user.email "correo@..."`	Tu correo (el de GVA) en los commits.
`gh auth login --with-token < auth.txt`	Inicia sesión en GitHub con el token guardado en auth.txt. El **<** mete el contenido del fichero como entrada del comando.
`gh auth setup-git`	Hace que git use esa sesión para hacer push sin pedir contraseña.

El token debe tener marcados todos los permisos (sobre todo **repo** y **workflow**; sin workflow no se puede subir el .yml de Actions).

1. Comandos de terminal que se usan

`mkdir carpeta`	Crea una carpeta.
`mkdir -p a/b/c`	Crea la carpeta y las intermedias que falten (**p** = parents). No da error si ya existe.
`cd carpeta / cd ..`	Entra en una carpeta / sube un nivel.
`ls`	Lista los ficheros (ls -a también muestra los ocultos, como .git).
`echo "texto" > fichero`	Crea el fichero con ese texto. **>** sobrescribe todo.
`echo "texto" >> fichero`	Añade el texto al final del fichero. **>>** no borra lo que había.
`git log --oneline > log.txt`	Guarda la salida de un comando en un fichero (así se hace log.txt).
`cat fichero`	Muestra el contenido del fichero.
`nano fichero`	Editor de texto: Ctrl+O guarda (Enter), Ctrl+X sale.
`unzip fichero.zip`	Descomprime el zip.

2. Comandos de Git

`git init -b main`	Convierte la carpeta en repo y llama **main** a la rama (el examen lo exige).
`git remote add origin URL`	Enlaza el repo local con el de GitHub. **origin** es el nombre del remoto. Sin remoto no hay push.
`git remote -v`	Muestra los remotos configurados (para comprobar la URL).
`git remote remove origin`	Borra el remoto (si te equivocaste de URL).
`git remote set-url origin URL`	Cambia la URL del remoto sin borrarlo.
`git status`	Estado: ficheros modificados, preparados, y si vas adelantado respecto al remoto.
`git add fichero`	Prepara un fichero para el commit. (git add . o git add * añade todo; el examen pide a veces "solo" un fichero.)
`git commit -m "mensaje"`	Guarda los cambios preparados con ese mensaje. El mensaje debe ser **exactamente** el del enunciado.
`git push -u origin main`	Sube main a GitHub. **-u** enlaza la rama local con origin/main (upstream); así luego basta con **git push**.

`git push origin main`	Igual pero sin -u: funciona, pero la próxima vez tendrás que escribirlo entero.
`git log --oneline`	Historial de commits, uno por línea.
`git reset --soft HEAD~1`	Deshace el último commit y deja los cambios preparados.
`git reset --hard HEAD~1`	Borra el último commit y sus cambios (peligroso).
`git update-ref -d HEAD`	Borra el commit si es el único que hay (HEAD~1 no existe).
`git commit --amend -m "msg"`	Cambia el mensaje del último commit.

Del temario (pueden entrar en el examen)

`git clone URL`	Descarga un repo de GitHub a tu equipo.
`git tag v1.0`	Crea una etiqueta ligera en el commit actual.
`git tag -a v1.0 -m "msg"`	Crea una etiqueta anotada (con mensaje).
`git push origin v1.0`	Sube una etiqueta (git push --tags sube todas).
`git branch nombre`	Crea una rama.
`git switch nombre`	Cambia de rama (switch -c nombre la crea y cambia).
`git merge nombre`	Fusiona la rama indicada en la rama actual.

3. Ejercicios paso a paso

Ejercicio 1 — crear antes el repo exercici-1 en GitHub (público)

```
mkdir ~/examen
cd ~/examen
git init -b main
git remote add origin https://github.com/jaisormar/exercici-1.git
echo "# Respostes — Jaime Sorolla" > respostes.md
git add respostes.md
git commit -m "Primer commit"
git push -u origin main
echo "Jaime Sorolla Martínez" > Readme.md
git add Readme.md
git commit -m "Segon commit"
git log --oneline > log.txt
git add log.txt
git commit -m "Tercer commit"
git push
```

Ojo: el enunciado pide push solo tras el primer commit, pero conviene dejar todo subido al final.

Ejercicio 2 — carpeta nueva = repo nuevo (otro git init y otro remote)

```
mkdir ~/examen/exercici-2
cd ~/examen/exercici-2
git init -b main
git remote add origin https://github.com/jaisormar/exercici-2.git
nano index.html            # contenido del enunciado, con tu nombre
git add index.html
git commit -m "Afegeix pàgina principal"
echo "# Capçalera" > Readme.md
git add Readme.md
git commit -m "Afegeix readme"
git push -u origin main
```

Ejercicio 3 — .gitignore

```
unzip exercici-3.zip
cd exercici-3
git init -b main
git remote add origin https://github.com/jaisormar/exercici-3.git
echo "logs/" > .gitignore
git add .gitignore
git commit -m "Afegeix .gitignore"
git push -u origin main
git add script.py
git commit -m "Afegeix script"
git push
```

.gitignore lista lo que Git debe ignorar; con **logs/** la carpeta logs no se sube nunca.

Ejercicio 4 — GitHub Pages (repo jaisormar.github.io)

```
# Si ya existía el repo, borrarlo y crearlo de nuevo (público)
unzip exercici-4.zip
cd exercici-4
git init -b main
git remote add origin https://github.com/jaisormar/jaisormar.github.io.git
git add .
git commit -m "Subir todo"
git push -u origin main
```

En GitHub: **Settings** → **Pages** → **Source: Deploy from a branch** → **main / (root)** → **Save**. Cuando esté publicada (1-2 min), el último paso:

```
echo "Jaime Sorolla Martínez" >> README.md
git add README.md
git commit -m "Afegeix el meu nom"
git push
```

Usa **>>** para añadir al final (con > borrarías el README). Aquí es **README.md** en mayúsculas.

Ejercicio 5 — GitHub Actions (repo exercici-5)

```
unzip exercici-5.zip
cd exercici-5
git init -b main
git remote add origin https://github.com/jaisormar/exercici-5.git
mkdir -p .github/workflows
nano .github/workflows/tests.yml     # el YAML de abajo
git add .
git commit -m "Afegeix workflow"
git push -u origin main
```

Luego, pestaña **Actions** del repo: los 3 jobs deben salir en verde (el de macOS puede tardar en arrancar).

4. El fichero tests.yml explicado

Tiene que estar en **.github/workflows/** (GitHub solo ejecuta los workflows de esa carpeta). El nombre tests.yml es libre. En YAML la **sangría indica qué va dentro de qué**: usa espacios (nunca tabuladores), 2 por nivel.

```yaml
name: Tests                       # nombre del workflow
on:                               # CUÁNDO se ejecuta
  push:                           #   cuando hay un push
    branches: [main]              #   a la rama main
jobs:                             # QUÉ se ejecuta: lista de jobs
  test-win:                       # job 1 (nombre exacto del enunciado)
    runs-on: windows-latest       #   sistema operativo donde corre
    steps:                        #   pasos del job, en orden
      - uses: actions/checkout@v6        # descarga el código del repo
      - uses: actions/setup-python@v6    # instala Python
        with:                            #   opciones de este paso
          python-version: "3.12"         #   versión (Windows = 3.12)
      - run: python -m unittest -v       # ejecuta los tests
  test-linux:                     # job 2
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"         # Linux y Mac = 3.13
      - run: python -m unittest -v
  test-mac:                       # job 3
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: python -m unittest -v
```

`name`	Nombre que aparece en la pestaña Actions.
`on: push: branches`	Evento que dispara el workflow: push a main.
`jobs`	Conjunto de trabajos; cada uno corre en su propia máquina, en paralelo.
`runs-on`	Sistema: windows-latest, ubuntu-latest o macos-latest.
`steps`	Lista de pasos; cada paso empieza con un guion (-).
`uses`	Usa una action ya hecha (checkout, setup-python).
`with`	Parámetros de la action (aquí, la versión de Python).
`run`	Ejecuta un comando de terminal.

5. Errores típicos (los que te salieron)

`Unknown command: '# Capçalera'`	Faltaba **echo** delante del texto.
`HEAD~1 revisión desconocida`	Solo hay un commit; no existe uno anterior.
`Repository not found`	URL mal escrita, o el repo aún no existe en GitHub / no es público.
`remoto origin ya existe`	Usa git remote set-url, o git remote remove origin y vuelve a añadirlo.
`git commit "mensaje" falla`	Falta el **-m**: git commit -m "mensaje".
`nano fichero < git log ...`	Mal: para guardar la salida de un comando usa **>**.

Consejo: antes de acabar, comprueba en GitHub que cada repo es público, que los mensajes de commit son idénticos al enunciado y que no te falta ningún push (git status).